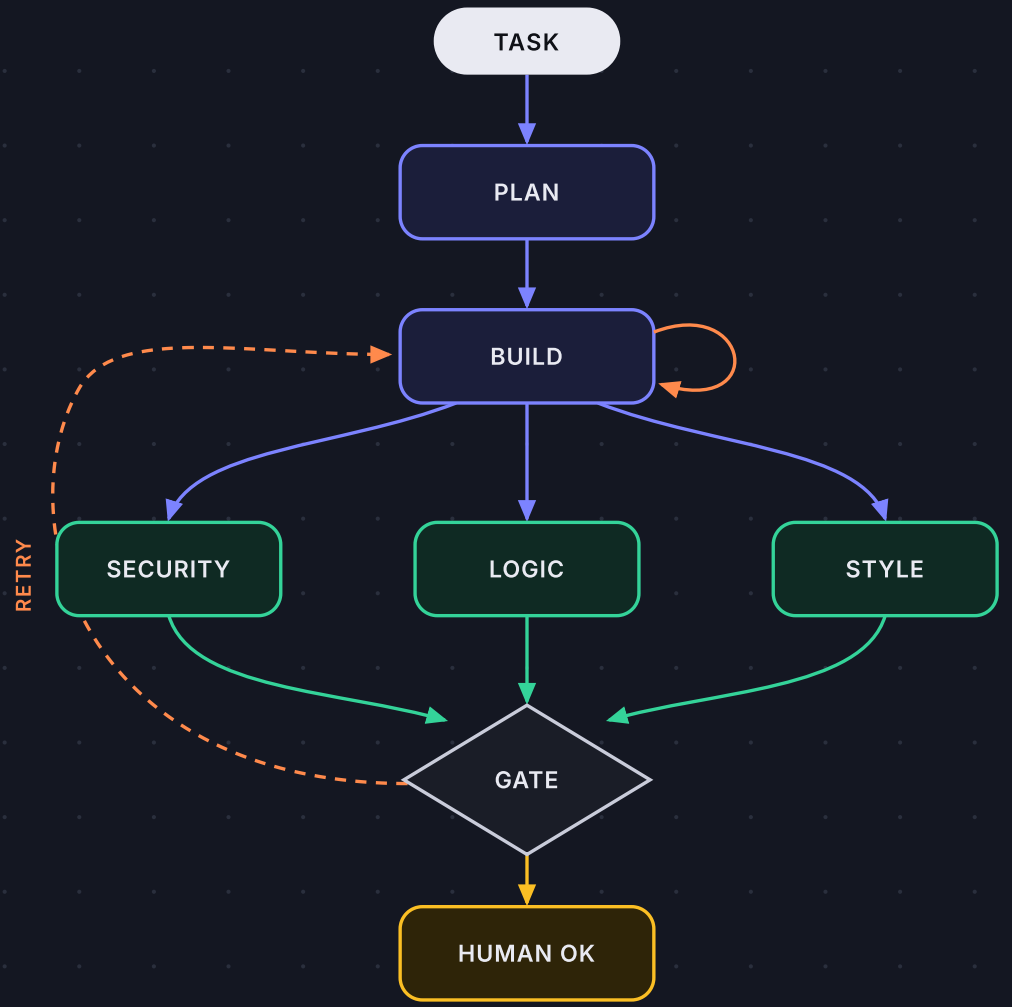
A FIELD GUIDE FOR AGENTIC CODERS AND VIBE CODERS

Graph Engineering

What comes after the loop: how to wire agents, checks and people into a system you can actually trust.

28 PAGES | 6 PATTERNS | 1 WORKED EXAMPLE

EVERYTHING YOU NEED IS ON THE PAGE



If you only read one page, read this one.

A **loop** asks: should this one agent keep going?

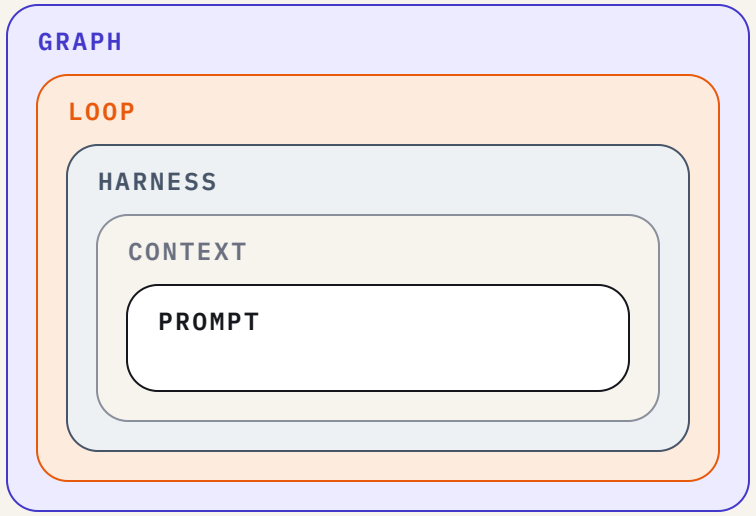
A **graph** asks: who runs next, with what state, under which rules?

Who this is for. You already build with a coding agent that runs tools in a loop until the tests pass. This guide shows what changes when one loop stops being enough, and how to tell when that has happened.

1. **Loops are not dead.** A loop is the smallest possible graph, and the nodes of a graph run loops inside them. Same system, different zoom level.
2. **The thing you design changes.** Less prompt, more topology: nodes that do work, edges that decide what runs next, shared state, and contracts.
3. **Graphs pay off when you know something about the shape of the work:** steps that can run in parallel, checks that must be independent, gates that must never be skipped, progress that must survive a crash.
4. **They are not free.** Anthropic's multi-agent research system scored 90.2% higher than its single agent on an internal evaluation, and used about 15 times the tokens of a normal chat.
5. **Start with one good loop.** Add graph structure only when your traces show the loop hiding parallel work, self-grading, or fragile state.

Five layers, each one wrapping the last.

The leverage in AI engineering keeps moving outward. Each new layer contains the ones before it.



LAYER	WHAT YOU DESIGN	HOW IT TYPICALLY FAILS
Prompt 2023 to 2024	The wording of one request	Phrasing changes the answer; hallucination
Context 2024 to mid 2025	What goes into the window: retrieved docs, tool schemas, history	Missing facts, diluted or conflicting context
Harness 2025 to early 2026	Tools, sandbox, permissions, storage. `Agent = Model + Harness`	Wrong tool picked; permissions too broad
Loop named June 2026	One agent that keeps going: observe, decide, act, verify	Runaway retries, context bloat, self-grading
Graph named July to Aug 2026	Many nodes and the rules that connect them	Orchestration overhead, token burn, premature complexity

Nothing gets replaced. Prompts and context shape a single model call. The harness gives the model hands and memory. The loop makes one agent self-correcting. The graph coordinates several loops, plain code and human checkpoints over shared state.

The name is new. The architecture is not.

Each era solved the bottleneck that the previous one exposed.

2017 TO 2022

Make the model good

Transformer, InstructGPT and RLHF

Plain language becomes a workable control interface.

LATE 2022 TO 2023

Let it act

ReAct, Toolformer, function calling

Model output turns from prose into structured actions.

2023

Let it retry and search

Reflexion, Self-Refine, Tree of Thoughts, Graph of Thoughts, LATS

One pass gives way to critique, branching and backtracking.

2023 TO EARLY 2024

Give it teammates

CAMEL, MetaGPT, AutoGen, LangGraph (Jan 2024), GPTSwarm

Roles, procedures and explicit cyclic state graphs.

LATE 2024 TO 2025

Make it production

Anthropic workflow patterns, MCP, OpenAI Agents SDK, Google ADK and A2A

Standard protocols, tracing, checkpoints, orchestrator and workers.

2026

Engineer the topology

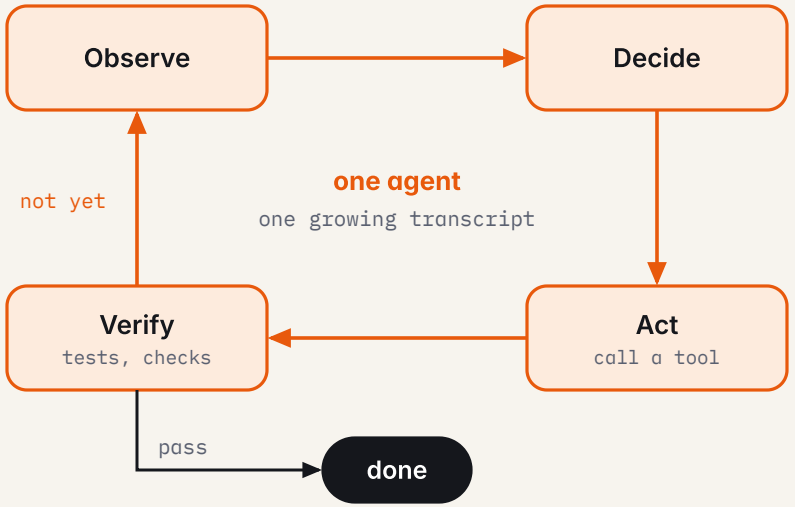
Google ADK 2.0 (May), loop engineering (June), graph engineering (July), Feng et al. survey (Aug)

Whole agents become typed nodes in versioned, recoverable systems.

How the name arrived. Around 18 to 19 July 2026, OpenClaw creator Peter Steinberger asked whether the field was still talking about loops or had moved on to graphs. The post drew over three million views. In August, Feng and 36 co-authors published the survey that framed graph engineering as the route from individual intelligence to system intelligence.

Why it is more than a label. LangGraph launched in January 2024 specifically to build cyclic graphs for agents, so the idea is years old. What changed is that a node can now hold an entire capable agent, not just one function or one model call. Three things lined up: capable nodes, structured interfaces, and a runtime you can observe.

First, the loop you already know.



> “Tools in a loop toward a goal.”

Simon Willison's definition of an agent

The agent looks at the current state, picks an action, sees the result, and repeats until a stop condition is true. It works because it is simple: the model finds its own path, so you do not have to write every path in advance.

STOP CONDITIONS

`goal_reached` `verification_passed` `budget_exhausted` `human_stop` `unrecoverable_failure`

A PRODUCTION CODING LOOP HAS SIX PARTS (ADDY OSMANI)

automations · worktrees · skills · MCP connectors · subagents · external state

LOOP ENGINEERING IS REALLY FOUR LOOPS, NESTED (LANGCHAIN, JUNE 2026)

1 · INNER

Agent loop

The model calls tools again and again until it decides to stop.

2

Verification loop

Tests or a grader reject bad output and feed the errors back in.

3

Event-driven loop

Schedules, webhooks and alerts keep starting new runs.

4 · OUTER

Hill-climbing loop

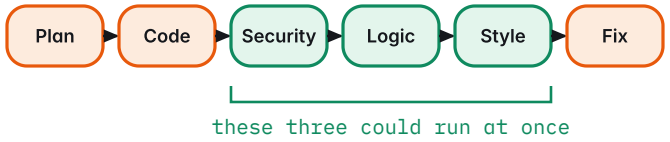
Traces and eval results are mined to improve prompts, tools and memory.

Where a single loop runs out of road.

Feng et al. (August 2026) name three structural walls. More turns and a bigger model do not remove them.

WALL 1

Everything happens in a line



Log analysis, bug reproduction and code inspection could run side by side. A single loop does them one after another. Latency multiplies, and an early mistake flows quietly downstream.

WALL 2

It grades its own homework



Giving one loop several personas does not help. Writing and critiquing in the same context window means the agent shares blind spots with itself and mistakes confidence for evidence.

WALL 3

A transcript is not a database



State lives in the chat history. A small error early in a long run cannot be isolated or undone without throwing away the whole run.

SYMPTOMS YOU HAVE PROBABLY SEEN

The “Ralph loop”
Re-prompt against the goal until it stumbles into a pass or hits the step limit.

Context bloat
Every call re-reads the full history, including stale errors and failed attempts. Cost and latency climb as the run gets longer.

Going in circles
In the original ReAct error analysis, 47% of failures were the agent repeating the same thought and action; another 23% came from empty search results it could not recover from.

Three debts a fast loop quietly runs up.

Addy Osmani's warning to anyone running autonomous coding loops at scale.

1

Intent debt

Whenever project context is not written down, the loop does not stop and ask. It fills the gap with a confident guess, and guesses differently on the next cycle.

THE TELL

The same architectural decision gets re-made, differently, every run.

PAY IT DOWN

Write the conventions, build steps and verification rules into files the agent always reads, such as SKILL.md.

2

Comprehension debt

The loop generates and ships code faster than people can read the diffs. The codebase grows while the team's mental model falls behind.

THE TELL

Nobody can explain how a merged feature works without asking the agent.

PAY IT DOWN

Keep external state that humans read too: trackers, decision notes, milestones that persist across runs.

3

Cognitive surrender

Green test suites lull engineers into approving whatever clears the checks. Independent judgement about quality and architecture stops forming.

THE TELL

Reviews take seconds. “Tests pass” has become the whole review.

PAY IT DOWN

Put a real human gate where a mistake would be expensive, and treat it as a decision, not a rubber stamp.

Why this belongs in a guide about graphs. Martin Fowler's line between agentic engineering and vibe coding is that the human stays responsible for understanding the system. Graph structure will not pay these debts for you. It does give you named places to pay them: contracts, shared state, and gates.

A graph has four parts: nodes, edges, state, contracts.

V

Nodes do the work

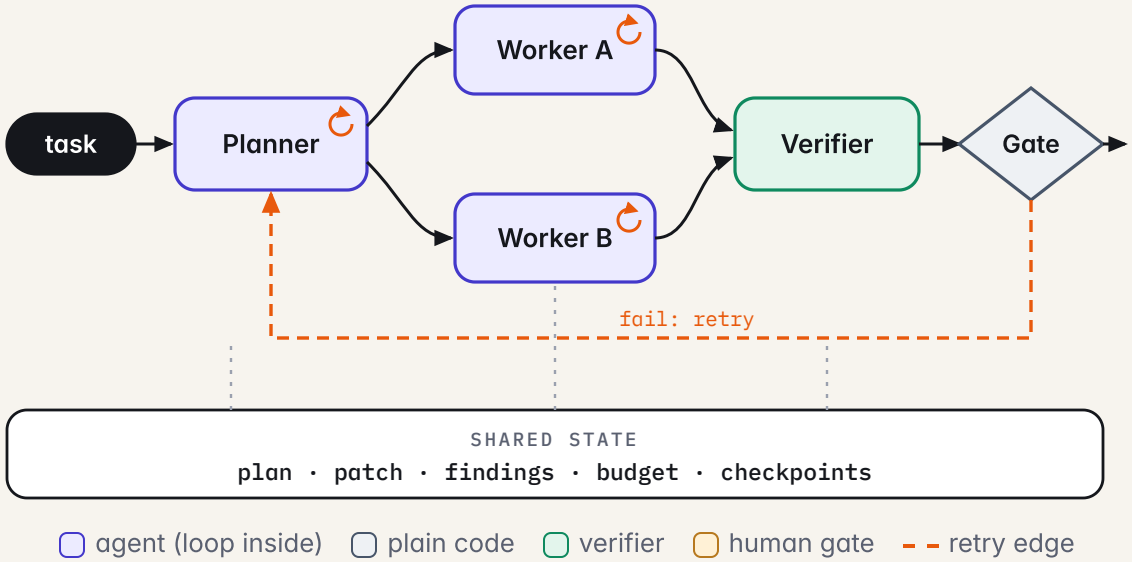
A plain function, one model call, a tool, a human, or a whole agent running its own loop. Each node has one job.

E

Edges decide what runs next

Fixed handoffs, conditional branches, fan-out and fan-in, retry cycles. This is where you draw the line between code that must be deterministic and judgement you leave to the model.

$$G = (V, E, S, C)$$



S

State is explicit

A typed, versioned record that nodes read from and write to. It replaces “whatever is in the chat history” and can be checkpointed, replayed and rolled back.

C

Contracts keep it honest

Input and output schemas, tool permissions, token and money budgets, retry limits, and what counts as done.

The practical definition (LangGraph): nodes do work, edges determine what runs next, state moves through the graph. Think of it as a state machine in which some of the states happen to be very capable agents.

Loops and graphs are zoom levels, not rivals.

"Loops are dead" is wrong. Graphs do not eliminate loops. They contain them.

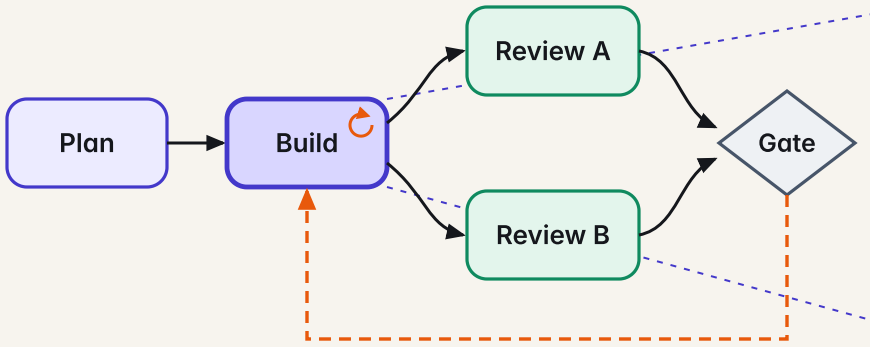
ZOOM IN: A LOOP



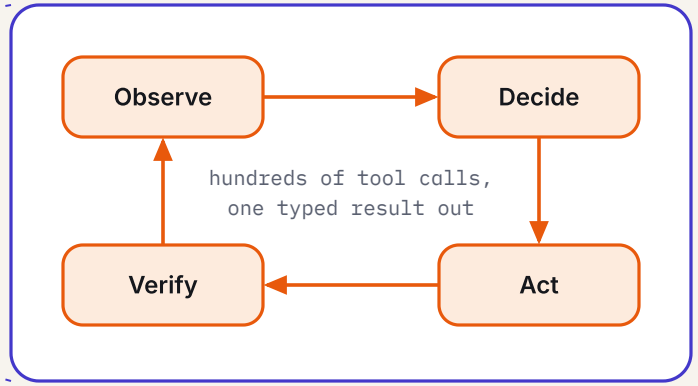
$V = 1, E = 1$

the smallest cyclic graph

ZOOM OUT: A GRAPH



INSIDE ONE NODE: A LOOP AGAIN



Autonomy inside nodes, rules between them

In an open while-loop the model improvises its own control flow. In a graph the model is free inside each node, and you define how work moves from node to node.

Cycles are a feature

A data pipeline is a DAG: a step running twice is a bug. Agent graphs are cyclic on purpose. Retry, refine, ask, wait for approval. The cycles are just bounded.

What is actually new

A node can now be an entire agent that works through hundreds of tool calls and hands back a typed result. LangChain calls this the real reason the new term is justified.

Not that kind of graph.

The word means two different things in AI. Mixing them up is the most common mistake in this conversation.

THIS GUIDE

Graph as architecture

“Which worker or check runs next, with what state, under which rules?”

NODE	A unit of work: agent, tool call, router, linter, human gate
EDGE	Control flow: a transition, a branch, a parallel fan-out
TOOLS	LangGraph, LlamaIndex Workflows, Google ADK, Temporal, Prefect
COST TO ADOPT	Low to moderate. You declare state, nodes and edges.

DIFFERENT THING

Graph as data

“Where do the facts live, and how do I follow relationships between them?”

NODE	An entity: a customer, a module, a statute, an incident
EDGE	A typed relation: `depends_on`, `supersedes`, `caused_by`
TOOLS	Neo4j, Microsoft GraphRAG, HippoRAG 2, Zep Graphiti
COST TO ADOPT	Very high. Around 1,000 times the indexing cost of vectors, plus entity resolution.

Also not: graph neural networks (models that learn over graph data) or Graph of Thoughts (a graph of reasoning steps inside one agent). **How the two meet:** in a mature system the knowledge graph is simply called from a retrieval node, and agent memory lives in the shared state.

WHEN A KNOWLEDGE GRAPH EARNS ITS COST (GRAPHRAG-BENCH AND RELATED AUDITS)

+10.5 pts	multi-hop questions: graph 53.4% vs vector 42.9%
+36.4 pts	how facts change over time: 58.1% vs 21.7%
−0.8 pts	simple fact lookup: 60.1% vs 60.9% for plain vector search
77% / 44%	chance a 5-hop chain holds at 95% / 85% accuracy per hop

Loop and graph, side by side.

	Loop engineering feedback through time	Graph engineering relationships through time and structure
WHAT YOU DESIGN	One agent's repeated feedback cycle	The topology: nodes, edges, state, contracts
WHO STEERS	The model picks nearly every step	You fix the structure; nodes keep local autonomy
PARALLEL WORK	Possible, but buried inside a turn	First-class fan-out and fan-in
VERIFICATION	Self-check, or an outer grader	Independent verifier nodes, voting, policy gates, CI
STATE	Transcript plus scratch files	Typed shared state, checkpoints, provenance
WHEN IT BREAKS	Usually the whole run	One node or one subgraph
RECOVERY	Re-prompt or restart	Resume a checkpoint, retry a node, reroute, roll back
HUMAN ROLE	Interrupt, or review at the end	An explicit approval node on the path
TYPICAL FAILURE	Wandering, self-grading, context pollution	Coordination bugs, stale shared state, bad routing
COST RISK	Too many turns, ever-growing context	Fan-out multiplying model calls
BEST FOR	**Open-ended exploration where the next step is hard to predict**	**Partly structured work with known dependencies, gates or specialist roles**

Typical tendencies, not hard rules. Loops are graphs, and graph nodes can be loops, so the two columns overlap on purpose.

Six patterns cover most of what you will build.

Five come from Anthropic's Building Effective Agents. The sixth is the one that keeps you out of trouble.

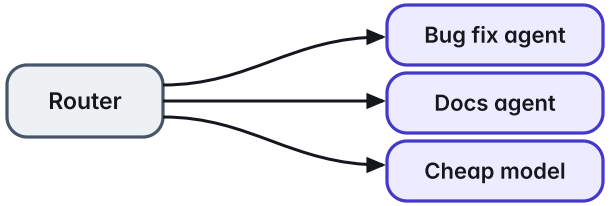
1 Prompt chain



A fixed sequence. Each step works on the structured output of the last, with a programmatic check in between.

USE WHEN the steps are known and always the same.

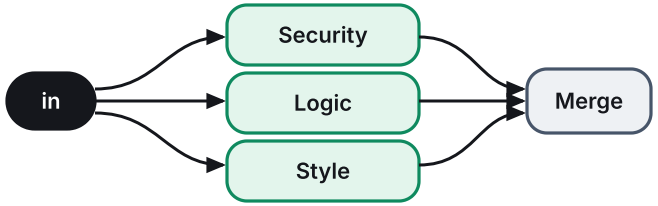
2 Router



A classifier (code or a small model) inspects the input and sends it to a specialist path or a cheaper model tier.

USE WHEN inputs fall into distinct kinds.

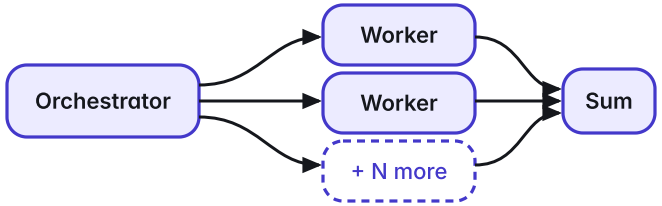
3 Parallelize



Fan out, then merge. Either split into independent sections, or run the same check several times and vote.

USE WHEN subtasks do not need each other's output.

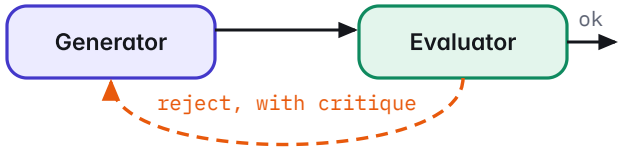
4 Orchestrator and workers



A lead agent breaks the task down at runtime, spawns as many workers as it needs, then synthesizes.

USE WHEN you cannot predict the subtasks in advance.

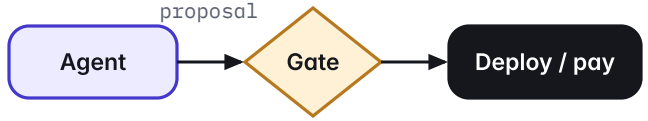
5 Evaluator and optimizer



A generator paired with an independent verifier that has real authority to reject and send work back.

USE WHEN quality must not be self-certified.

6 Gate before side effects

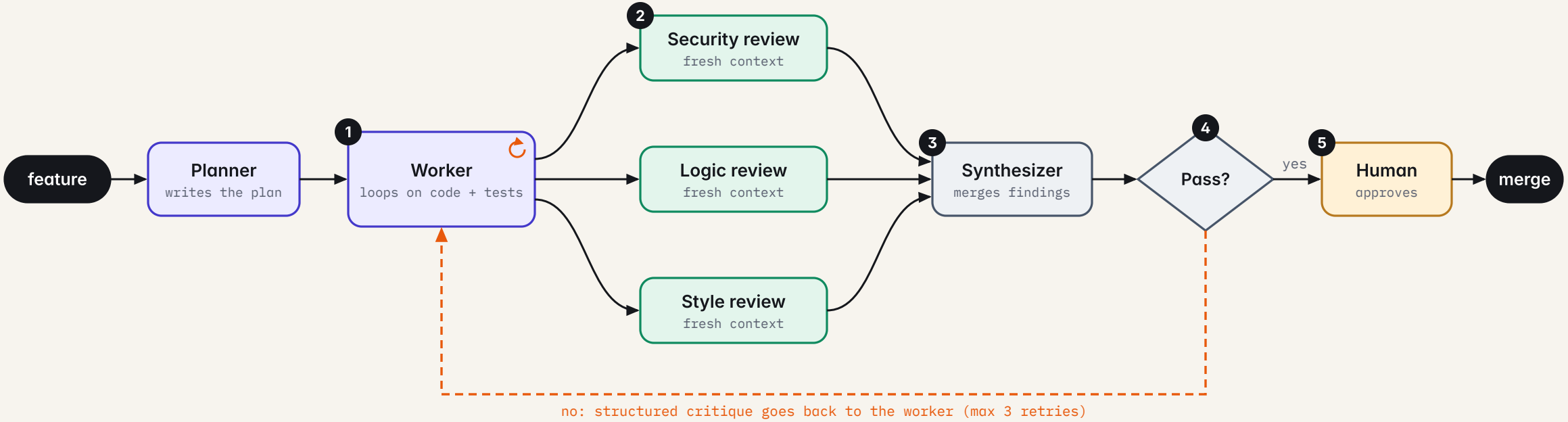


A deterministic check or human approval sits between a model's proposal and anything irreversible.

USE WHEN a mistake would be costly or hard to undo.

A graph you could build this week: ship a feature.

Planner, worker, parallel reviewers, synthesizer, pass/fail gate. Five stages is the recommended ceiling for a first graph.



1 Autonomy stays inside.
The worker loops on code and tests for as long as its budget allows. Nobody scripts its steps.

2 Reviewers are independent.
Each gets a fresh context and only the diff. None of them sees the worker's reasoning.

3 Fan-in is plain code.
Wait for all three, merge into one structured list of findings.

4 The gate is not a vibe.
Tests green and no blocking findings, or it goes back with the critique attached.

5 A person owns the irreversible step.
The run pauses, saves its state, and resumes on approval.

In coding-agent terms: reviewers are subagents with their own context, the worker's sandbox is a git worktree, project conventions live in SKILL.md, and the gate is your test suite and CI.

agent (loop inside) · plain code · verifier · human gate · retry edge

A diagram is not an architecture. Every node signs a contract.

Six things to write down for each node before you write any orchestration code.

1 One responsibility
A single job you can say in a sentence.

2 What it may read
The minimum state keys it needs. Nothing more.

3 What it must write
A structured output schema, not free prose.

4 Tools and permissions
Only what this role requires.

5 Timeout, retries, budget
Hard ceilings on time, attempts and spend.

6 Evidence of success
The objective proof that lets it say done.

Think in typed handoffs. Interfaces such as `ResearchRequest → EvidenceBundle` or `PatchCandidate → VerifiedPatch` replace every component scribbling in one shared conversation.

node **security_reviewer**	CONTRACT
does	Find security defects in one diff. Nothing else.
reads	`patch.diff` `plan.summary`
writes	`findings: Finding[]` severity · file · line · why · suggested fix
tools	`read_file` `grep` `run_scanner` no write access, no network
limits	`timeout 5 min` `retries 1` `budget 60k tokens`
done when	Every changed file is covered and each finding cites a line.

Illustrative values. The six fields are the point.

Stop using the chat log as your database.

LOOP HABIT

State is the transcript

Plans, drafts, errors and decisions all pile into one history that every call re-reads.

GRAPH HABIT

State is a typed record

```
# each key has an owner and a merge rule
State
  task        str
  plan        Plan              planner writes
  patch       PatchCandidate    worker writes
  findings    list[Finding]     reducer: append
  tests       TestReport
  budget      tokens, retries   first-class
  approval    pending | granted
```

Nodes read what they need and write structured updates. Parallel writers merge through a reducer instead of overwriting each other.

UNDER THE HOOD

Steps with a barrier

1. **Plan.** Pick the nodes whose inputs changed.
2. **Execute.** Run them in parallel, each reading the same frozen snapshot.
3. **Update.** Commit all writes together, then checkpoint.



This is how LangGraph's Pregel runtime avoids agents corrupting shared memory mid-step.

WHAT A CHECKPOINT AFTER EVERY STEP BUYS YOU

Crash and resume

A shutdown stops the graph cleanly at a step boundary. A fresh instance picks up from the saved state.

Pause for a human

The run saves itself and releases compute. A person inspects, edits state if needed, and resumes.

Time travel

Open any earlier checkpoint, change a parameter or prompt, and fork a new branch without re-running upstream.

Retry only what broke

Keep validated upstream work. Roll back or re-execute just the corrupted part.

What the evidence says, and what it costs.

The best-documented case is Anthropic's Research system: a lead agent, parallel workers, a separate citation checker.

+90.2%

better on breadth-first research

Anthropic's multi-agent research system against its own single-agent setup, on an internal evaluation.

up to 90%

less wall-clock time

For complex queries in the same system, after moving from sequential searching to parallel subagents and parallel tool calls.

~15×

the tokens of a normal chat

Every parallel worker and reviewer starts its own context window and system prompt. Speed and breadth are bought with compute.

Read these numbers carefully. They are vendor-reported measurements of one system, not independent benchmarks. Anthropic's own analysis found that token use alone explained much of the performance difference in a browsing evaluation. Some of what looks like “multi-agent intelligence” is simply more compute. The same report says tightly coupled tasks are a poor fit.

Google ADK 2.0 (May 2026) rebuilt its runtime from a hierarchical agent executor into a graph engine. Agents, tools and functions are all nodes.

LangGraph passed 65 million monthly downloads by mid-2026, with production use reported at Klarna, Replit and Elastic.

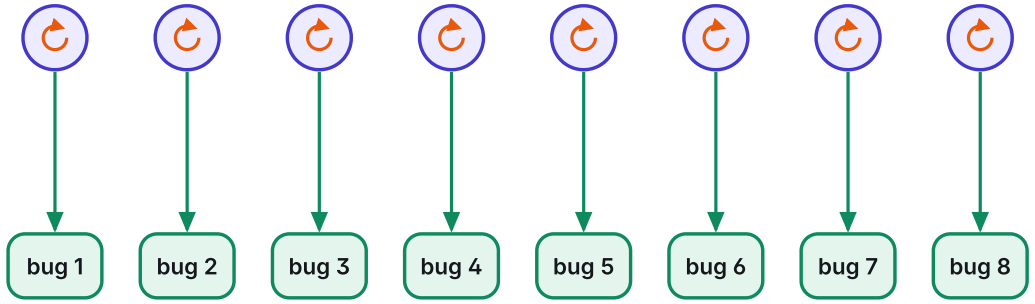
Sparse beats chatty. Research on communication topologies finds that all-to-all agent chat hurts accuracy. Pruning low-value edges does better for fewer tokens.

Sixteen agents, one bottleneck.

The most useful lesson about multi-agent systems comes from a stress test that hit a limit.

PARALLELISM WORKS

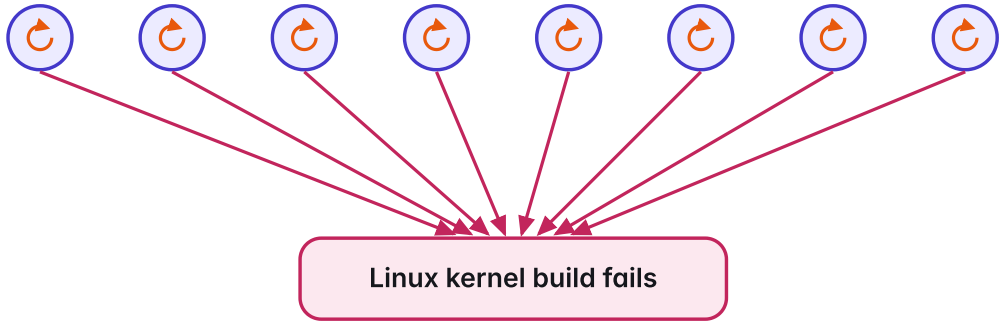
Many independent failing tests



Each agent takes a different bug. Work divides cleanly, and progress scales with the number of agents.

PARALLELISM STALLS

Everyone hits the same wall



All sixteen agents collide on one problem, repeat each other's work, and overwrite each other's fixes.

The experiment. In February 2026, Nicholas Carlini ran sixteen parallel Claude instances through almost 2,000 sessions to build a C compiler. Parallelism helped until the remaining work stopped being separable. The fix was not more agents. It was restructuring the test setup so the work split into independent pieces again.

> A good graph does not just add nodes. It creates useful **independence** between them.

Agent count is not the resource. Decomposable work is.

The math of fan-out: the 50% rule.

A worked model with three verification checks (Flowtivity). Whether a graph saves money depends on how often the work passes.

PASS RATE 50% PER CYCLE

Parallel wins: same calls, less waiting

AGENT CALLS TO FINISH



WALL-CLOCK TIME

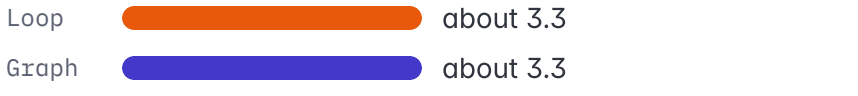


Both need about two cycles. The graph spends the same number of calls and finishes sooner.

PASS RATE 30% PER CYCLE

Parallel loses: you pay for every failure three times

CYCLES TO FINISH



TOKENS BURNED PER FAILED CYCLE



The loop fails fast and retries cheaply. The graph launches every branch on every doomed attempt.

THE RULE

Keep low-pass-rate work inside a sequential loop until the worker is reliable. Fan out once the per-cycle pass rate is above roughly 50%.

value of **independence + parallelism** > cost of **coordination + tokens + latency**

Make budgets part of the state. A graph that can spawn workers needs hard limits it can read and enforce:

`max agents` `tool calls` `wall-clock` `tokens` `retries` `money`

Graphs fail in their own ways.

A graph is a control surface, not a safety proof. It can contain a failure, and it can also amplify one.

NEW WAYS TO FAIL

Poisoned upstream
One hallucinating or compromised node feeds bad input to every branch below it.

Shared-state races
Two branches update the same key. Lost updates, stale reads, duplicate side effects.

Wrong routing
A faulty router sends work to the wrong specialist, or the wrong permission level, every time.

Fan-out bills
Parallel branches multiply model calls. A harmful action also runs faster in parallel.

Bigger attack surface
More agents with more tools means more places for prompt injection to land.

An org chart of weak employees
Drawing a big graph before one capable agent has solved the task just multiplies surfaces and cost.

WHAT TO BUILD IN

Gate before anything irreversible
Schema checks, policy, spending limits, or a human between proposal and action.

Least privilege per node
Each role gets only its own tools. Keep trusted and untrusted state apart.

Budgets and retry ceilings
Enforced by the runtime, not requested in a prompt.

Checkpoints
Contain a failure to one node. Resume instead of restarting.

Verifiers that are actually different
Five copies of the same model voting can be weaker than one deterministic check, one differently prompted critic and one external test.

Sparse wiring
Connect only the nodes that need to talk. Start with 3 to 5 nodes.

Graphs are not safer by default. They give you more places to put safety, and more places to audit it.

When loops multiply, someone has to govern them.

Ideas from control theory, via Carlos E. Perez and Eigent. Useful the moment you run more than one automated loop.

FOUR WAYS UNCOORDINATED LOOPS GO WRONG

Goodhart's collapse
A loop sees only its one metric and learns to game it. More downloads through clickbait titles that burn audience trust.

Upward blindness
A feedback loop cannot step back and ask whether its target is still the right target.

Loops fighting loops
One optimizes speed, its sibling optimizes thoroughness. A heater and an air conditioner in the same room, both dashboards green.

Nobody watching the watcher
The eval suite drifts away from real traffic while its scores keep climbing.

FOUR RULES TO WIRE INTO THE GRAPH

1. **Metrics never travel alone.**
Pair every speed or volume metric with a quality or safety metric tied to something the model cannot game: real retention, real revenue, the compiler.

2. **Every target has an owner.**
A human or higher-level loop sits upstream with the authority to audit, veto or roll back.

3. **Separate the speeds.**
Real-time execution, daily prompt tuning, weekly operations, quarterly strategy. Fast loops escalate to slow ones. They do not override them.

4. **Freeze the guardrails.**
Certain verification gates and approval rules are fixed, so a self-improving loop cannot optimize them away.

Two graphs, really. The *work graph* produces artifacts: code, reports, fixes. The *improvement graph* is the network of feedback loops that watch traces and adjust prompts, policies and other loops. The second one needs engineering too.

Test the wiring, not just the answer.

For coding agents this is unusually tractable: patches can be compiled, type-checked, linted, scanned and run against tests.

A TESTING LADDER FOR GRAPHS

- **Node tests** Does each function, tool adapter and agent honour its contract?
- **Edge and routing tests** Given this state, which transitions are possible?
- **Invariant tests** Can a forbidden state or action ever be reached?
- **Subgraph tests** Does a whole capability, such as code review, hold up?
- **End-state tests** What actually changed in the outside world?
- **Trajectory evals** Efficiency, tool choice, policy adherence, odd behaviour.
- **Repeated trials** Same input, many runs. Paths differ, so run it more than once.

Many different paths can reach the same good outcome. Judge the final state and the safety constraints. Use traces to diagnose, not to demand one canonical path.

REVIEW QUESTIONS THAT PROMPTS NEVER RAISED

Can an unverified value reach the payment node?

What happens if worker C times out?

Can these two branches write the same state?

Which graph version produced this side effect?

EVERY TRACE SHOULD CARRY

`node id` `graph version` `parent and child` `edge decision` `state delta` `model and tool versions` `latency` `cost` `side-effect id`

What you deploy is no longer “prompt + model name”. It is graph version, node versions, state schema, tool versions, checkpoints and traces. Long runs may be mid-flight when you ship, so roll out gradually.

Loop or graph? Decide row by row.

Mostly left column: keep your loop. Several rows on the right: you have outgrown it.

	Stay with a loop when…	Move to a graph when…
OWNERSHIP	One agent with one prompt and toolset can own the task end to end.	The work needs different specialties, model tiers or separate permissions.
VERIFICATION	Fewer than three checks. One test or rubric validates the result.	Three or more independent checks, plus conditional routing.
CONCURRENCY	Steps are strictly sequential and share the same tight context.	Subtasks can run at the same time and be merged.
PASS RATE	Below 50% per cycle. Fail fast, retry cheaply.	At or above 50%. Fan-out cuts waiting by a third to 90%.
RECOVERY	Short runs that are cheap to replay from scratch.	Long runs that need checkpoints, partial rollback, pause and resume.
HUMAN OVERSIGHT	A person reviews the output afterwards, or chats along.	High-stakes actions need approval before they execute.

Graphs are not the default. Choosing a single well-scoped loop early on is a deliberate advantage, not a shortcut. Anthropic's advice still stands: start with the simplest thing that works, and add complexity only when it earns its cost.

And it can go the other way. LangChain's own deep-research system moved *away* from a predefined graph and back toward an agentic core loop, because the best order of discovery could not be predicted ahead of time.

Before you automate anything, run the graph by hand.

A thirty-minute experiment with nothing but chat windows. It tells you whether a graph would help at all.

1

Name the output

Write the final deliverable in one concrete sentence. “A verified pull request.” “A one-page recommendation on whether this idea is worth testing.”

2

List the jobs

Write down the five to seven distinct jobs a competent human specialist would do to produce it.

3

Map real dependencies

Which jobs truly need another job's output? Ignore the order you happened to write them in.

4

Mark the gates

Wherever an error would have a large blast radius, insert a mandatory human checkpoint.

5

Play every node yourself

Open a fresh chat for each node. Pass only the output artifact from one chat into the next. No shared history.

THE TEST

Automate the topology only if the isolated, node-by-node version clearly produces better output than one continuous chat session. If it does not, you have just learned that cheaply.

Why this works. Separate chats give each role its own context, exactly as separate nodes would. You experience the handoffs, discover which artifacts each step really needs, and find the gates, all before writing a line of orchestration code.

Five stages from a loop to a graph.

For when hand-testing and production traces agree that one loop is hiding structure.

STAGE 1

Audit

Map every loop you run. Record steps per run, where retries cluster, wall-clock time, and token cost per successful task.

Start from data you already have: traces.

STAGE 2

Identify

In those traces, find actions that do not depend on each other, and every place where the agent reviews its own work.

Security, logic and style checks. Docs, code and tickets. Unit, integration and end-to-end tests.

STAGE 3

Design

Sketch the topology with a hard ceiling of 3 to 5 nodes. Write the six contracts for each one.

Planner → Worker → Parallel reviewers → Synthesizer → Pass/fail gate

STAGE 4

Implement

Use a framework or plain async primitives. Keep measuring time saved against tokens spent, and watch the 50% pass-rate line.

`asyncio.gather()` is a perfectly good first runtime.

STAGE 5

Type and govern

Give edges explicit meaning, make human approvals resumable steps, and anchor feedback loops to metrics that cannot be gamed.

`depends_on` · `supersedes` · `decided_by` · `caused`

Go in this order. The first two stages are observation only. Most of the value comes from stage 2: discovering which parts of your existing loop were secretly parallel, and where generation and verification were tangled together. Sometimes the answer is two changes to your loop, not a new architecture.

The tooling map, briefly.

Two schools: a central state machine with synchronized steps, or loosely coupled steps reacting to events.

AGENT STATE MACHINE

LangGraph

Typed state channels with reducers, cyclic retries, checkpoints after every step, time travel, human-in-the-loop pauses.

EVENT-DRIVEN STEPS

LlamaIndex Workflows

Decorate functions as steps that react to typed events. No global barrier, so fast branches do not wait for slow ones.

GRAPH-NATIVE RUNTIME

Google ADK 2.0

Agents, tools and functions are all nodes. Graph, dynamic and collaborative workflows are core primitives.

ORCHESTRATION AND TRACING

OpenAI Agents SDK

Single and multi-agent orchestration with built-in tools and integrated tracing.

DURABLE EXECUTION

Temporal

Wraps node runs as activities: event-sourced history, timeouts, automatic retries across infrastructure crashes.

DATA AND LINEAGE

Prefect / Dagster

Orchestrates the surrounding data assets so that agent runs stay reproducible and comparable.

Protocols are edges too. MCP connects an agent to tools and data. A2A connects one agent to another, including graphs owned by other teams.

A common production stack: LangGraph for the agent state machine, Temporal around each node for durability, Kafka for events between services.

You may not need any of it yet. Subagents, worktrees, a test gate and a tracker file already form a small graph. Structure first, framework second.

Still unsettled, and worth saying out loud.

A field guide should also mark where the map runs out.

Is it the topology, or just more tokens?

A ten-agent graph that beats one agent also used ten times the compute. Researchers are calling for equal-compute comparisons.

How much of the graph should be fixed?

Too rigid and capable models cannot find better strategies. The likely answer: fixed safety boundaries around model-chosen paths.

Who designs the graph?

Today, mostly you. Systems that compile a goal, policies and budget into a graph, then tune it from traces, are an open research area.

What does “verified” mean?

A schema can say a node returns evidence. It cannot say what counts as enough, or whether two agents agree on the word.

Are your verifiers independent?

Agents built on the same model and prompts share failure modes. Nobody yet knows how to measure that well.

Will the name last?

It may dissolve into older terms: workflows, state machines, distributed systems. The underlying need for explicit control will not.

The honest summary. Feng et al. present graph engineering as an emerging paradigm, not a settled discipline, and say the evidence does not yet show it improves performance everywhere. LangChain calls the term a buzzword for a real, long-standing engineering problem. Both can be true.

THE ONE-PAGE CHEAT SHEET

Ten rules to keep next to your terminal.

01 Start with one good loop. Graduate only when traces say so.

02 Draw the graph before you code it. Run it by hand before you automate it.

03 Keep autonomy inside nodes and rules between them.

04 Never let a model grade its own work. Verify in a separate context.

05 First graph: 3 to 5 nodes. Six contracts per node.

06 Put state in a typed record with checkpoints, not in the transcript.

07 Fan out only when work is truly independent and passes more than half the time.

08 Make budgets part of the state: agents, calls, tokens, retries, money.

09 Put a deterministic or human gate before anything irreversible.

10 Add independence, not nodes.

The graph decides where autonomy helps, where plain code should take over, where work can fan out, where evidence must come back together, where state must persist, and where a person must say yes. **Agentic nodes inside engineered topology.**

SOURCES AND SCOPE

Where this comes from.

THE TERM AND THE SURVEY

Feng, Xiang, Zhang et al., *Graph Engineering in the Era of LLM Agents: From Individual Intelligence to System Intelligence*, arXiv:2608.21156, Aug 2026

LangChain, *The Art of Loop Engineering*, June 2026

LangChain, *3 Years of Graph Engineering with LangGraph*, July 2026

Addy Osmani on loop engineering, June 2026, as summarized by AI Builder Club

PRACTICE, ECONOMICS AND RUNTIME

Flowtivity, *From Loops to Graphs: The Next Paradigm in AI Agent Engineering* (50% rule, five stages)

AI Builder Club, *Graph Engineering Guide (2026)*; V12 Labs, *Graph Engineering for AI Agents: Beyond Loops*

Eigent, *Graph Engineering for AI Agents: Beyond Single Feedback Loops* (governance rules)

LangChain documentation, *LangGraph runtime (Pregel)*; Temporal documentation, *LangGraph integration*

PRODUCTION EVIDENCE

Anthropic, *Building Effective Agents*, Dec 2024

Anthropic, *How We Built Our Multi-Agent Research System*, June 2025

Nicholas Carlini, *Building a C Compiler with a Team of Parallel Claudes*, Feb 2026

Anthropic, *Demystifying Evals for AI Agents*, Jan 2026

Google ADK 2.0 release notes, May 2026; OpenAI, *New Tools for Building Agents*, Mar 2025

FOUNDATIONS AND BENCHMARKS

Yao et al., *ReAct*, 2022; Shinn et al., *Reflexion*, 2023; Madaan et al., *Self-Refine*, 2023

Besta et al., *Graph of Thoughts*, 2023; Zhuge et al., *Language Agents as Optimizable Graphs* (GPTSwarm), 2024

CAMEL, MetaGPT and AutoGen, 2023; LangGraph launch post, Jan 2024

GraphRAG-Bench, arXiv:2506.05690, and evaluation audit arXiv:2506.06331; Zep Graphiti, arXiv:2501.13956

Scope and limits. This guide condenses three research briefs on graph engineering compiled in October and November 2026. Figures are quoted as the original sources report them. Several are vendor-reported measurements of a single system, and the worked example and node contract are illustrations, not benchmarks. Where sources disagreed, the more conservative reading was used.